From Concept to Application

Activity week 7-8

Presented by

Harvey M. Bulatao

From Block

2.4 BSIT

Subject

Datastructure

Submitted to

Darwin Vargas

Date

Oct 5, 2026

SCREENSHOTS:

```
--- STARTING REFACTORED SUBSYSTEM ---
Added item: 10
Added item: 20
Added item: 30
Added item: 40
Added item: 50
Added item: 60
```

```
Transposed Matrix:
1 4 7
2 5 8
3 6 9
```

```
--- STARTING REFACTORED SUBSYSTEM ---
Added item: 10
Added item: 20
Added item: 30
Added item: 40
Added item: 50
Added item: 60

Size: 6
Capacity: 10
Current List Contents: 10 20 30 40 50 60
Found 30 at index: 2
Item removed from index 2
Current List Contents: 10 20 40 50 60
Invalid index!
Current List Contents: 10 20 40 50 60
Invalid index!
Current List Contents: 10 20 40 50 60
Search for 999: -1

Transposed Matrix:
1 4 7
2 5 8
3 6 9

--- SUBSYSTEM FINISHED ---
```

```
Size: 6
Capacity: 10
Current List Contents: 10 20 30 40 50 60
Found 30 at index: 2
Item removed from index 2
Current List Contents: 10 20 40 50 60
Invalid index!
Current List Contents: 10 20 40 50 60
Invalid index!
Current List Contents: 10 20 40 50 60
Search for 999: -1
```

REFLECTION:

1. Experiential Reflection

The most challenging defect to locate was the memory leak during dynamic array resizing. At first, the program still worked, so the problem was not easy to notice. Through systematic unit testing, I was able to test different situations and identify that the old array was not properly deleted after resizing. Compared to random debugging, unit testing made it easier to find the exact function and condition causing the problem.

2. Architectural Evaluation

For a system that requires frequent insertions in the middle of a list, I would recommend using a linked list. Unlike a dynamic array, a linked list does not need to reallocate the whole buffer when adding elements. It can insert an element by changing the links between nodes. This can make frequent mid-list insertions more efficient, although it uses extra memory for pointers.

3. Professional Growth

Reviewing another programmer's code helped me understand the importance of writing clean and organized C++ code. I learned that small problems like missing bounds checking, memory leaks, and unclear naming can cause bigger problems later. For my upcoming Milestone 1 project, I will try to use better naming, proper memory management, clear functions, and more testing before considering my code finished.